Company: Amazon

Interview Type: TECHNICAL

Q1: Optimizing Warehouse Retrieval Robotics

Difficulty: Hard

Tags: Binary Search, Greedy, Sliding Window

Problem Statement: Amazon is designing a robotic system to retrieve packages from a long conveyor belt. The belt has N packages, where `packages[i]` represents the weight of the i-th package. You have K identical robots. Each robot can pick up a contiguous sequence of packages. However, there is a constraint: the total weight any single robot carries cannot exceed its capacity W, and the difference between the heaviest and lightest package picked by any single robot cannot exceed a stability threshold S. Find the minimum possible capacity W such that all N packages can be retrieved by K robots given a fixed stability threshold S.

Expected Approach: A naive approach would use recursion with memoization (Dynamic Programming) to try all possible split points, resulting in $O(N^2 \cdot K)$ complexity, which is too slow for large N.

Optimized Approach: Use Binary Search on the answer (the capacity W). The range for binary search is `[max(packages), sum(packages)]`. For a mid-value capacity mid, use a Greedy approach with a Two-Pointer/Sliding Window and a Monotonic Queue (or TreeMap) to check if N packages can be covered by K robots. The greedy check ensures each robot takes as many packages as possible without violating the weight sum mid and the stability threshold S.

Time Complexity: $O(N \log(\sum \text{weights}))$

Space Complexity: $O(N)$ to store package data and monotonic queues.

Optimizations: Use two monotonic deques to track the min and max within the current sliding window in $O(1)$ to validate the stability threshold S.

Hints: If you can solve it for capacity W, can you solve it for $W+1$? This monotonicity suggests binary search. Use the greedy property: always extend a robot's range as far as possible.

Q2: AWS Network Connectivity Resilience

Difficulty: Medium-Hard

Tags: Graph Theory, Tarjan's Algorithm, Depth First Search

Problem Statement: An AWS VPC consists of N microservices (nodes) and M bidirectional network connections (edges). A "Critical Link" is defined as a connection that, if severed, would split the VPC into two or more disconnected components, interrupting service. Given the network topology, identify all Critical Links to prioritize them for redundancy upgrades. Return the IDs of the connected services for each critical link.

Expected Approach: For every edge, temporarily remove it and run a BFS/DFS to check if the graph remains connected. This results in $O(E \cdot (V + E))$ time complexity, which is inefficient for large scale VPCs.

Optimized Approach: Use Tarjan's Bridge-Finding Algorithm (or equivalent discovery/low-link value logic). Perform a single DFS traversal. For each node, maintain a `discovery_time` and a `low_link_value` (the smallest discovery time reachable from that node in the DFS tree, including back-edges). An edge (u, v) is a critical link if the `low_link_value` of v is strictly greater than the `discovery_time` of u, meaning there is no alternative path from v back to u or its ancestors.

Time Complexity: $O(V + E)$

Space Complexity: $O(V + E)$ for the adjacency list and recursion stack.

Optimizations: Use an adjacency list instead of a matrix. Handle parallel edges between the same two services, as they prevent a link from being critical.

Hints: Think about the properties of a cycle. If an edge is part of a cycle, is it still a critical link? How can discovery times help identify back-edges?

Q3: Real-time Trending Products

Difficulty: Medium

Tags: Sliding Window, Hash Map, Doubly Linked List

Problem Statement: Amazon's analytics engine needs to track the top K most viewed products within a rolling 60-minute window. Design a data structure that supports two operations:
1. `recordView(productId, timestamp)`: Records a product view at a specific time.
2. `getTopK()`: Returns the K most viewed products in the last 60 minutes.
The stream of timestamps is non-decreasing.

Expected Approach: Use a Hash Map to store product frequencies and a Min-Heap to store the top K. For the sliding window, use a Queue to store `(productId, timestamp)` pairs. When `recordView` or `getTopK` is called, poll expired entries from the Queue and decrement their counts in the Hash Map. Rebuild the heap for every `getTopK` call.

Optimized Approach: Use a Hash Map for `productId` to frequency and a specialized "Frequency List" (similar to an LFU Cache). This consists of a Doubly Linked List where each node represents a frequency count and contains a set of product IDs. When a count increases, move the product to the next frequency node. To handle the sliding window, use a Queue to track arrival times. When an entry expires, move the product to the previous frequency node in $O(1)$. Maintain the top K by traversing the Frequency List from tail to head.

Time Complexity: $O(1)$ for `recordView` (amortized), $O(K)$ for `getTopK`.

Space Complexity: $O(N)$ where N is the number of views in the 60-minute window.

Optimizations: Use a Balanced BST if K is large and updates are frequent, or maintain a constant-size pointer to the top elements in the Frequency List to avoid full traversals.

Hints: How can we modify the LFU (Least Frequently Used) cache architecture to support both incrementing and decrementing counts in constant time?

Q4: Warehouse Robot Battery Management

Difficulty: Hard

Tags: BFS, Dijkstra, State-space Search

Problem Statement: A warehouse robot moves on an $N \times M$ grid with obstacles. The robot has a maximum battery capacity C. Every move to an adjacent cell consumes 1 unit of battery. Certain cells contain charging stations that instantly refill the battery to C. Given a starting point, a target, and a list of charging stations, find the minimum number of steps to reach the target without the battery ever falling below 0.

Expected Approach: Standard BFS to find the shortest path. However, this fails because the shortest path might not be feasible due to battery constraints, and a longer path through a charging station might be necessary.

Optimized Approach: Model the problem as a state-space search. Each state is defined as $(x, y, current\_battery)$. Use a 3D array `visited[N][M][C+1]` to keep track of the minimum steps taken to reach cell (x, y) with a specific battery level. Run a BFS (since each step is weight 1) starting from (sx, sy, C). If the current cell is a charging station, the battery level in the next state resets to C. If $(x, y, battery)$ has been reached before with a battery level $\ge$ current battery, prune the path.

Time Complexity: $O(N \times M \times C)$

Space Complexity: $O(N \times M \times C)$

Optimizations: Use Dijkstra's algorithm if moving between cells has different costs (e.g., different floor surfaces). Use a bitset for the battery dimension if C is small to reduce memory footprint.

Hints: Think of the battery level as a third dimension in your graph. What is the minimum state needed to ensure we don't revisit inefficient paths?

Q5: Critical Supply Chain Connections

Difficulty: Hard

Tags: Graph, DFS, Tarjan's Algorithm, Bridges

Problem Statement: Amazon's fulfillment network is represented as an undirected graph where nodes are warehouses and edges are transit routes. A "Critical Connection" is defined as a route that, if removed, would result in at least one warehouse becoming unreachable from the others. Given n warehouses and a list of `connections`, return all critical connections in any order.

Expected Approach: Use a standard Depth First Search (DFS) to traverse the graph. For each node, track the discovery time (when it was first visited) and the low-link value (the lowest discovery time reachable from that node, including back-edges in the DFS tree).

Optimized Approach: Implement Tarjan's Bridge-finding algorithm. During DFS, if a child node `v` cannot reach a node with a discovery time less than or equal to its parent `u`'s discovery time (`low[v] > disc[u]`), then the edge `(u, v)` is a bridge.

Time Complexity: O(V + E), where V is the number of warehouses and E is the number of connections.

Space Complexity: O(V) for the discovery and low-link arrays and the recursion stack.

Optimizations: Use an adjacency list instead of a matrix. Avoid re-visiting the immediate parent in the DFS to prevent trivial cycles.

Hints: Think about how cycles affect connectivity; if an edge is part of a cycle, it cannot be a critical connection.

Q6: Optimal Bundle Selection

Difficulty: Medium

Tags: Monotonic Stack, Greedy, Array

Problem Statement: To optimize shipping, you must select a "Competitive Bundle" of k products from a sequence of n product IDs. A bundle is more competitive if it is lexicographically smaller than another. Given an integer array `productIds` and an integer k, return the most competitive subsequence of size k.

Expected Approach: Use a greedy strategy to pick the smallest available product ID at each step. However, you must ensure that there are enough remaining elements in the array to complete a bundle of size k.

Optimized Approach: Utilize a monotonic increasing stack. Iterate through the `productIds`. While the current ID is smaller than the stack's top and the number of remaining elements plus the current stack size is greater than k, pop from the stack. Push the current ID if the stack size is less than k.

Time Complexity: O(N), where N is the length of `productIds`, as each element is pushed and popped at most once.

Space Complexity: O(k) for the stack storing the result.

Optimizations: Pre-calculate the number of elements that can be safely discarded ($n - k$) to simplify the loop condition.

Hints: If you see a smaller element, you want to replace a larger previous element, but only if you have enough elements left to fill the k slots.

Q7: Robot Route Optimization with Battery Constraints

Difficulty: Hard

Tags: Graph, Dijkstra, State-space Search

Problem Statement: An automated robot in an Amazon fulfillment center needs to move from a starting cell `(sr, sc)` to a target cell `(tr, tc)` on an `M x N` grid. The grid contains obstacles (1 for obstacle, 0 for empty). The robot has a maximum battery capacity `B`. Moving to an adjacent cell consumes 1 unit of battery. Some empty cells are "Charging Stations" that instantly refill the battery to `B`. If the battery level reaches 0, the robot cannot move further unless the current cell is a charging station. Find the minimum number of steps to reach the target.

Expected Approach: A standard BFS would only track `(row, col)`, which is insufficient because a longer path might be necessary to reach a charging station.

Optimized Approach: Use Dijkstra's algorithm where the state is defined as `(row, col, current_battery)`. Maintain a 3D array `dist[row][col][battery]` to store the minimum steps taken to reach a cell with a specific battery level. Priority Queue stores `(steps, r, c, b)`. When moving to an adjacent cell, decrement battery. If the new cell is a charging station, reset battery to `B`.

Time Complexity: O(M * N * B * log(M * N * B))

Space Complexity: O(M * N * B)

Optimizations: Use a 2D array `max_battery[row][col]` to store the highest battery level seen so far for that cell at a lower or equal step count to prune states.

Hints: Think of this as a shortest path problem on a layered graph where each layer represents the remaining battery.

Q8: Fulfillment Stream "Flapping" Detection

Difficulty: Medium-Hard

Tags: Hash Map, Prefix Sums, Sliding Window

Problem Statement: Amazon's monitoring system tracks "Success" (S) and "Failure" (F) events from a delivery service as a string. A "Balanced Window" is defined as a contiguous segment where the ratio of Success to Failure is exactly `K:1`. Given a string of events (e.g., "SSFSFFS") and an integer `K`, find the length of the longest Balanced Window.

Expected Approach: Brute force checking all subarrays takes $O(N^2)$, which is too slow for high-frequency logs.

Optimized Approach: Transform the problem using prefix sums. Let `countS` be the number of 'S' and `countF` be the number of 'F'. The condition `countS / countF = K` can be rewritten as `countS - K * countF = 0`. Iterate through the string, maintaining a running value `val = (current_S_count) - (K * current_F_count)`. Store the first occurrence of each `val` in a Hash Map. If the same `val` is encountered again at index `i`, the subarray from `map[val] + 1` to `i` is balanced. Calculate the length and track the maximum.

Time Complexity: O(N)

Space Complexity: O(N)

Optimizations: Use a single pass and handle the case where `countF` is zero separately if `K > 0`.

Hints: Convert the ratio constraint into a linear equation that can be tracked with a single running total.

Q9: Multi-Level Warehouse Robot Pathfinding

Difficulty: Hard

Tags: Graph, Dijkstra, 3D-Grid

Problem Statement: An Amazon warehouse is represented as a 3D grid (Levels, Rows, Columns). A robot must move a package from source (l_1, r_1, c_1) to destination (l_2, r_2, c_2). The robot can move North, South, East, West on the same level (cost 1). Some cells contain "Gravity Chutes" which allow the robot to drop to the same (r, c) on the level below instantly (cost 0). Some cells

contain "High-Speed Conveyors" which move the robot 2 cells in a fixed direction (cost 1). Obstacles are present. Find the minimum cost.

Expected Approach: Model the warehouse as a weighted directed graph where each cell is a node. Use Dijkstra's algorithm to find the shortest path.

Optimized Approach: Use a Priority Queue (Min-Heap) for Dijkstra. Represent the grid as a 1D array or hash map for faster lookups. Pre-calculate conveyor destinations to handle jumps as single edges.

Time Complexity: $O(V \log V + E)$, where $V = L \times R \times C$ and $E \approx 6V$.

Space Complexity: $O(V)$ to store distances and the priority queue.

Optimizations: Use a 0-1 BFS variant if only weights 0 and 1 are present; otherwise, use a Fibonacci heap for theoretical $O(E + V \log V)$ performance.

Hints: Treat the Gravity Chutes and Conveyors as directed edges with unique weights. Ensure boundary checks for 2-cell conveyor jumps.

Q10: Efficient Customer Search Autocomplete

Difficulty: Medium-Hard

Tags: Trie, Min-Heap, Design

Problem Statement: Design a system for Amazon’s search bar that returns the top 3 most frequently searched terms starting with a given prefix. You are given a stream of search queries and their historical frequencies. The system must support an `update(term, count)` operation and a `query(prefix)` operation. If frequencies are tied, sort alphabetically.

Expected Approach: Use a Trie where each node stores a list of all words passing through it. For the query, traverse to the prefix node and perform a DFS to find all terms, then sort.

Optimized Approach: In each Trie node, maintain a pre-computed list of the "Top 3" terms. When `update` is called, traverse from the leaf to the root, updating the "Top 3" list at each node using a Min-Heap of size 3 to keep the operation $O(L \times 3 \log 3)$, where L is word length.

Time Complexity: $O(L)$ for both `update` and `query`, where L is the length of the string.

Space Complexity: $O(N \times L)$, where N is the number of unique terms and L is the average length.

Optimizations: Use a frequency map to handle updates to existing words efficiently before propagating changes up the Trie.

Hints: Don't re-sort the entire dataset on every query. Think about how to store "state" at each node to make queries $O(1)$ relative to the number of total terms.
